
MemEvolve: Meta-Evolution of Agent Memory Systems

Guibin Zhang^{*1} Haotian Ren^{*2} Chong Zhan² Junhao Wang³ He Zhu⁴ Wangchunshu Zhou⁵
Shuicheng Yan¹

Abstract

Self-evolving memory systems are rapidly reshaping the evolutionary paradigm of large language model (LLM)-based agents. Prior work has predominantly relied on manually engineered memory architectures to store trajectories, distill experience, and synthesize reusable tools, enabling agents to evolve on the fly within environment interactions. However, this paradigm is fundamentally constrained by the *staticity* of the memory system itself: while memory facilitates agent-level evolving, the underlying memory architecture cannot be meta-adapted to diverse task contexts. To address this gap, we propose **MemEvolve**, a meta-evolutionary framework that jointly evolves agents’ experiential knowledge and their memory architecture, allowing agent systems not only to accumulate experience but also to progressively refine how they learn from it. To ground **MemEvolve** in prior work and promote openness in future self-evolving systems, we introduce **EvoLab**, a unified memory codebase that distills twelve representative memory systems into a modular design space (*encode, store, retrieve, manage*), providing a standardized implementation substrate and a fair experimental arena. Extensive evaluations on four challenging agentic benchmarks show that **MemEvolve** delivers (i) substantial performance gains, improving frameworks such as SmolAgent and Flash-Searcher by up to 17.06%, and (ii) strong cross-task and cross-LLM generalization, yielding memory architectures that transfer effectively across diverse benchmarks and backbones. Codes are available at [here](#).

1. Introduction

Language agents and agent systems, empowered by increasingly capable foundation models (Team et al., 2025a;b) and sophisticated scaffolding (Wang et al., 2024a; LangChain, 2023), have advanced rapidly, demonstrating unprecedented performance across complex tasks such as deep research (Chen et al., 2025), scientific discovery (Bai et al., 2025; Wei et al., 2025b), and industrial report generation (Zhang et al., 2025g). A key driving force behind this success is the *agent memory system* (Zhang et al., 2024b; Hu et al., 2025c), which persistently captures interactions between the agent and environment, distilling them into diverse forms of knowledge and skills, and thereby enabling large language model (LLM)-based agents to evolve continuously in task solving and world exploration.

Naturally, the choice of memory paradigm plays a decisive role in shaping an agent’s capacity for on-the-fly self-evolution. Initial designs centered on raw trajectory storage and few-shot prompting (Zhong et al., 2024; Wen et al., 2024), which were later superseded by more abstracted textual artifacts such as tips, shortcuts, and reasoning templates (Ouyang et al., 2025; Zhang et al., 2025b; Ye et al., 2025; Tang et al., 2025). Recent advances have also explored structured tool interfaces (e.g., APIs (Zheng et al., 2025), MCPs (Qiu et al., 2025b;a; Zhang et al., 2025h)) and code-level repositories (Zhang et al., 2025e; Wang et al., 2025a) as memory carriers. Amid this growing diversity, an inquisitive practitioner might ask: *What kind of memory architecture most effectively drives agent self-improving?*

We posit that no universally optimal memory architecture exists. Memory systems tailored to distill reusable APIs from trajectories may excel in web-centric tasks yet offer limited benefit for mathematical or scientific reasoning, while self-critique-based memories, though effective in reasoning-intensive domains (Cai et al., 2025), often underperform in coding and tool-use settings (Zhang et al., 2025d). These trade-offs stem from the *static nature* of current memory pipelines, which are typically fixed in their ingestion, abstraction, and retrieval design (Zhang et al., 2025i). By contrast, effective human learners are capable of adapting their learning strategies to the task, e.g., prioritizing memorization for literary analysis while abstracting solution templates

¹National University of Singapore ²Beijing University of Posts and Telecommunications ³Chinese University of Hong Kong ⁴OPPO ⁵Bytedance Inc. . Correspondence to: Wangchunshu Zhou <chunshu@bytedance.com>, Shuicheng Yan <yansc@nus.edu.sg>.

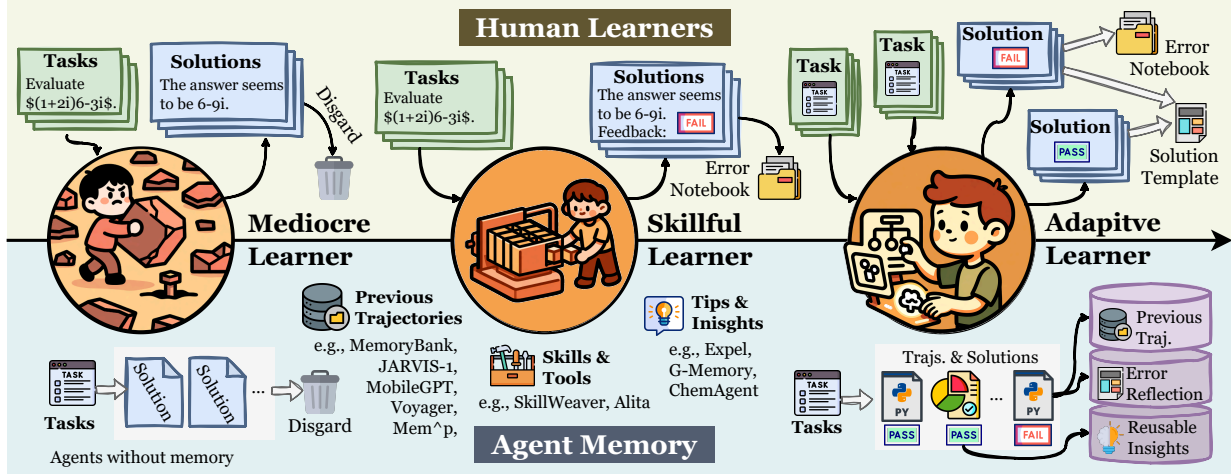


Figure 1. The paradigm of agent self-evolution admits a natural analogy to human learning. At one extreme, a *mediocre learner* fails to benefit from experience (agents without memory). More capable *skillful learners* can extract reusable skills from past experience, albeit through a pre-defined abstraction scheme. In contrast, an *adaptive learner* simultaneously accumulates experience and dynamically adjusts the strategy by which experience is consolidated and utilized. This final regime precisely characterizes the objective of MemEvolve.

for mathematics. We argue that agent memory systems must similarly transition from merely *skillful* to *adaptive*, enabling the memory architecture itself to *meta-evolve* in response to task demands while preserving generalizability, as shown in Figure 1. To put it more formally:

How can a memory system not only facilitate the agent system’s evolution but also *meta-evolve its own architecture* to achieve superior task-domain performance gains while preserving generalizability?

To address the challenge, we introduce **MemEvolve**, a framework that facilitates the dual evolution of an agent’s experience and its memory architecture. Conceptually, **MemEvolve** operates as a bilevel optimization process: the inner loop performs a *first-order evolution*, where the agent, guided by a fixed memory system, adapts to a continuous stream of new tasks by populating its experience base. The outer loop drives a *second-order evolution*, meta-learning a more effective memory architecture to accelerate future learning. This allows the agent not only to evolve, but to evolve more efficiently and intelligently over time.

However, the vast and heterogeneous memory design space (e.g., knowledge graphs, skill libraries, vector databases) poses a major challenge for controllable optimization. To make this tractable, we introduce a modular decomposition of memory architectures into four components: ♣ *Encode* (experience perception and formatting), ♦ *Store* (information commitment), ♥ *Retrieve* (context-aware recall), and ♠ *Manage* (consolidation and forgetting). **MemEvolve** evolves the programmatic implementations of these modules in a model-driven manner using inner-loop performance feedback, forming a virtuous cycle in which improved architectures enhance agent learning, and stronger agents yield

higher-quality trajectories that sharpen the outer-loop fitness signal for subsequent evolution.

To ground our framework within the diverse landscape of existing self-improving agent memories, we systematically re-implement twelve representative architectures in a unified modular design space, including EXPEL (Zhao et al., 2024), AGENT WORKFLOW MEMORY (Wang et al., 2024b), and DYNAMIC CHEATSHEET (Suzgun et al., 2025). The resulting framework, denoted as **EvoLab**, serves both as an empirical foundation for MemEvolve’s evolutionary process and as a standardized codebase to facilitate future self-evolving research. Our contributions are as follows:

- ❶ **Unified Codebase:** We introduce **EvoLab**, a modular design space for self-improving agent memory systems encompassing four key components (*encoding*, *storage*, *retrieval*, and *management*), providing unified implementations and benchmark support for a wide range of prevailing agent memory systems.
- ❷ **Meta-Evolution Framework:** We propose **MemEvolve**, a meta-evolutionary framework that jointly evolves both agents’ experiential knowledge and their underlying memory architecture, in which agent systems not only accumulate experience but also progressively refine their mechanism for learning from it.
- ❸ **Experimental Evaluation:** Extensive experiments on four challenging agentic benchmarks demonstrate that **MemEvolve** delivers (I) **substantial performance gains**, improving frameworks such as SmolAgent and Flash-Searcher by up to 17.06%; and (II) **cross-domain, cross-framework and cross-LLM generalization**, where memory systems evolved on TaskCraft yield 2.0–9.09% gains with unseen benchmarks and backbone models.

2. Related Work

LLM Agent Systems. Recent work has rapidly advanced LLM-based agent systems along two axes (Tran et al., 2025; Fang et al., 2025a). In terms of **system complexity**, research has evolved from single-agent workflows with manual tool orchestration (Wu et al., 2023; Significant-Gravitas, 2023) to multi-agent frameworks with automated coordination and rich MCP integration (Zhang et al., 2024a; 2025a; Wang et al., 2025b; Zhang et al., 2025c). Along the **task domain** axis, capabilities have expanded from coding and mathematical reasoning (Hong et al., 2024; Yin et al., 2023) to deep research and scientific discovery (Du et al., 2025; Ghaireeb et al., 2025). A growing set of open-source systems now achieve competitive performance on challenging benchmarks such as (Mialon et al., 2023; Phan et al., 2025; Wei et al., 2025a; Chen et al., 2025), including OWL (Hu et al., 2025a), CK-PRO (Fang et al., 2025c), AGENTORCHESTRA (Zhang et al., 2025f), and AIME (Shi et al., 2025b).

Agent Memory Architectures. Agent memory systems are commonly categorized into *personalized* and *self-improving* memories (Zhang et al., 2024b; Hu et al., 2025c), where the former captures user-specific preferences and the latter distills knowledge and skills from continual interaction, which is the focus of this work. Early self-improving systems stored raw trajectories as few-shot exemplars (Wang et al., 2023; Zhong et al., 2024; Packer et al., 2023), while later approaches abstracted experience into higher-level insights, procedural guidance, and reusable tools or structured repositories (Yang et al., 2025; Sun & Zeng, 2025; Wu et al., 2025b; Wang et al., 2025c; Zheng et al., 2025; Fang et al., 2025b; Zhao et al., 2025; Qiu et al., 2025a;b; Zhang et al., 2025e). Despite representational differences, these methods share a common goal: enabling agents to learn, adapt, and improve through continual experience.

3. EvolveLab Codebase

In this section, we first formalize the LLM agentic system and its associated memory architecture, then present the modular design space of **EvolveLab**, and finally introduce the unified codebase **EvolveLab**.

3.1. Preliminary

We formalize an LLM-based agentic system as $\mathcal{M} = \langle \mathcal{I}, \mathcal{S}, \mathcal{A}, \Psi, \Omega \rangle$, where $\mathcal{I}$ indexes the $\{1, \dots, N\}$ agents, $\mathcal{S}$ denotes the shared state space, $\mathcal{A} = \bigcup_{i \in \mathcal{I}} \mathcal{A}_i$ represents the joint action space, and $\Psi(s_{t+1} | s_t, a_t, \mu(t))$ describes the environment dynamics with $\mu(t) \in \mathcal{I}$ indicating the active agent at time step t . The system leverages a memory module Ω , which maintains a continuously evolving memory state M_t . At each step, the active agent observes the current state s_t , considers a task-specific query Q , and interacts with Ω to retrieve contextually relevant memory

c_t , conditioned on its interaction history $\mathcal{H}_t$. The agent μ_t ’s policy π_{μ_t} then delivers an action:

$$a_t = \pi_{\mu(t)}(s_t, \mathcal{H}_t, Q, c_t), \quad c_t \sim \Omega(M_t, s_t, \mathcal{H}_t, Q). \quad (1)$$

Following task execution, a trajectory $\tau = (s_0, a_0, \dots, s_T)$ is recorded, with an overall performance evaluated via a terminal reward $R(\tau)$. The memory system assimilates new experience units ϵ , which can vary in granularity (from individual state-action transitions to aggregated segments or complete trajectories), and updates the memory state as $M_{t+1} = \Omega(M_t, \epsilon)$, where Ω abstracts the memory’s mechanisms for integrating and organizing new knowledge.

3.2. Modular Design Space of Memory Systems

The heterogeneous and rapidly evolving landscape of self-improving agent memories presents challenges for systematic analysis and controlled experimentation. To address this, we propose a modular design space that decomposes any memory system Ω into four functionally distinct yet interdependent components: $\Omega = (\mathcal{E}, \mathcal{U}, \mathcal{R}, \mathcal{G})$, representing *encode*, *store*, *retrieve*, and *manage* operations, respectively. ♣ **Encode** ($\mathcal{E}$): Transforms raw experiences, such as trajectory segments $\tau_t = (s_t, a_t, s_{t+1})$, tool outputs, or self-critiques, into structured representations $e_t = \mathcal{E}(\epsilon_t)$. Encoding may be as simple as compressing raw traces (Zheng et al., 2023) or as sophisticated as extracting generalizable lessons (Zheng et al., 2025). ♦ **Store** ($\mathcal{U}$): Integrates encoded experiences into the persistent memory M_t , yielding $M_{t+1} = \mathcal{U}(M_t, e_t)$. Storage can be vector databases (Zhao et al., 2024), knowledge graphs (Zhang et al., 2025b; Rasmussen et al., 2025), or others. ♥ **Retrieve** ($\mathcal{R}$): Provides task-relevant memory content, formalized as $c_t = \mathcal{R}(M_t, s_t, Q)$, which informs the agent’s policy decision a_t . Retrieved content may include reusable tools (Zhang et al., 2025f), planning experience (Tang et al., 2025), or distilled procedural knowledge (Wu et al., 2025b; Yang et al., 2025; Fang et al., 2025b). ♠ **Manage** ($\mathcal{G}$): Performs offline and asynchronous operations such as consolidation, abstraction, or selective forgetting to maintain long-term memory quality and efficiency, denoted as $M'_t = \mathcal{G}(M_t)$.

This modular abstraction allows us to represent each memory system as a specific combination of programmatic implementations for $(\mathcal{E}, \mathcal{U}, \mathcal{R}, \mathcal{G})$, forming a “genotype” that facilitates the meta-evolutionary process of **MemEvolve**.

3.3. EvolveLab Codebase

Based on the above design space, we introduce **EvolveLab**, a unified and extensible codebase designed for the systematic implementation and evaluation of self-evolving memories, serving as a standardized resource for the community.

Table 1. Self-improving memory systems implemented in **EvoLab**. In “Mul.” column, ♠ indicates support for single-agent settings, while ♠♠ denotes compatibility with multi-agent systems. “Gran.” specifies the granularity at which memory is provided (*step-wise* vs. *trajectory-wise*), and “Online” indicates whether memory is updated *on-the-fly* (🔄) or maintained as an offline repository (📦).

Method	Date	Mul.	Gran.	Online	Encode	Store	Retrieve	Manage
I. Voyager	2023.5	♠	traj.	🔄	Traj. & Tips	Vector DB	Semantic Search	N/A
II. ExpeL	2023.8	♠	traj.	🔄	Traj. & Insights	Vector DB	Contrastive Comparison	N/A
III. Generative	2023.10	♠♠	traj.	🔄	Traj. & Insights	Vector DB	Semantic Search	N/A
IV. DILU	2024.2	♠	traj.	🔄	Traj.	Vector DB	Semantic Search	N/A
V. AWM	2024.9	♠	traj.	🔄	Workflows	Vector DB	Semantic Search	N/A
VI. Mobile-E	2025.1	♠	step	🔄	Tips & Shortcuts	Vector DB	Semantic Search	N/A
VII. Cheatsheet	2025.4	♠	traj.	🔄	Tips & Shortcuts	JSON	Semantic Search	N/A
VIII. SkillWeaver	2025.4	♠	traj.	🔄	APIs	Tool Library	Function Matching	Skill Pruning
IX. G-Memory	2025.6	♠♠	traj.	🔄	Tips & Workflow	Graph	Graph/Semantic Search	Episodic Consolidation
X. Agent-KB	2025.7	♠♠	step	🔄	Tips & Workflow	Hybrid DB	Hybrid Search	Deduplication
XI. Memp	2025.8	♠	step.	🔄	Tips & Workflow	JSON	Semantic Search	Failure-driven Adjustment
XII. EvolveR	2025.10	♠	step.	🔄	Tips & Workflow	JSON	Contrastive Comparison	Update & Pruning

Implementation. The cornerstone of **EvoLab** is its modular and hierarchical design. Every memory architecture re-implemented in our codebase (see Table 1) inherits from a singular abstract base class, **BASEMEMORYPROVIDER**, which enforces the unified four-component interface: ♣ Encode, ♦ Store, ♥ Retrieve, and ♠ Manage. This ensures that diverse memory mechanisms can be managed, modified, and evolved under a consistent programmatic structure. The framework offers out-of-the-box support for multiple challenging benchmarks in both *online* and *offline* manner. More details are at Section A.

4. MemEvolve: Meta-Evolving Memory

4.1. Dual-Evolution Process

Traditional self-improving memory systems operate under a *fixed memory architecture*, where the memory interface Ω is predefined and remains static. Within this architecture, the agent iteratively populates and updates its memory state M_t through interaction with the environment and task experiences. For a trajectory τ induced by a query $\mathcal{Q}$, the memory evolution follows

$$M_{t+1} = \Omega(M_t, \epsilon_\tau), \quad \epsilon_\tau \in \mathcal{E}(\tau), \quad (2)$$

where $\mathcal{E}(\cdot)$ denotes an experience extraction operator that maps a trajectory to a set of experience units, and ϵ_τ is an element sampled from this set. While this enables the accumulation of knowledge, it fundamentally precludes *architectural adaptation*, as the memory interface Ω itself remains immutable.

To transcend this limitation, we propose a **dual-evolution process** that jointly evolves (i) the agent’s memory base and (ii) the underlying memory architectures (as illustrated in Figure 2). Instead of a single static Ω , we maintain, at each evolutionary iteration k , a finite set of candidate memory systems $\{\Omega_j^{(k)}\}_{j \in \mathcal{J}^{(k)}}$, where each $\Omega_j^{(k)}$ is instantiated as a concrete realization of the four-component memory interface $\Omega_j^{(k)} \triangleq (\mathcal{E}_j^{(k)}, \mathcal{U}_j^{(k)}, \mathcal{R}_j^{(k)}, \mathcal{G}_j^{(k)})$. The initial iteration start from a singleton set $|\mathcal{J}^{(0)}| = 1$, corresponding to a

hand-designed baseline memory, while later iterations admit multiple competing candidates. Given a batch of trajectories $\mathcal{T}_j^{(k)}$ independently generated by executing the agent with memory system $\Omega_j^{(k)}$, the dual-evolution process consists of two nested loops:

- **Inner Loop (Experience Evolution).** For each candidate memory system $\Omega_j^{(k)}$, the associated memory state $M_{t,j}^{(k)}$, initialized as an empty memory at the beginning of iteration k , is updated along trajectories $\tau \in \mathcal{T}_j^{(k)}$ via

$$M_{t+1,j}^{(k)} = \Omega_j^{(k)}(M_{t,j}^{(k)}, \epsilon_\tau), \quad \epsilon_\tau \in \mathcal{E}_j^{(k)}(\tau). \quad (3)$$

Executing the agent with $\Omega_j^{(k)}$ over $\mathcal{T}_j^{(k)}$ yields, for each trajectory τ , a feedback vector $\mathbf{f}_j(\tau) \in \mathbb{R}^d$, where $d = 3$ corresponds to the number of evaluation metrics (*i.e.*, task success, token consumption, and latency). An aggregation operator $\mathcal{S}$ summarizes the inner-loop outcomes for each candidate as

$$\mathbf{F}_j^{(k)} = \mathcal{S}(\{\mathbf{f}_j(\tau)\}_{\tau \in \mathcal{T}_j^{(k)}}), \quad j \in \mathcal{J}^{(k)}. \quad (4)$$

- **Outer Loop (Architectural Evolution).** The set of memory architectures is then updated based on the collection of summaries $\{\mathbf{F}_j^{(k)}\}_{j \in \mathcal{J}^{(k)}}$. A meta-evolution operator $\mathcal{F}$ selects high-performing candidates and proposes new variants, producing the next iteration’s candidate set:

$$\{\Omega_{j'}^{(k+1)}\}_{j' \in \mathcal{J}^{(k+1)}} = \mathcal{F}\left(\{\Omega_j^{(k)}\}_{j \in \mathcal{J}^{(k)}}, \{\mathbf{F}_j^{(k)}\}_{j \in \mathcal{J}^{(k)}}\right). \quad (5)$$

Specifically, $\mathcal{F}$ ranks candidates according to $\mathbf{F}_j^{(k)}$, retains the top- K memory systems, and generates new architectures by modifying or recombining all four components $(\mathcal{E}, \mathcal{U}, \mathcal{R}, \mathcal{G})$ of the selected candidates, where K denotes a fixed survivor budget. We detail the implementation of $\mathcal{F}(\cdot)$ in Section 4.2.

Unified view. At a higher level, each iteration k alternates between (i) evolving the *memory experience base* from an

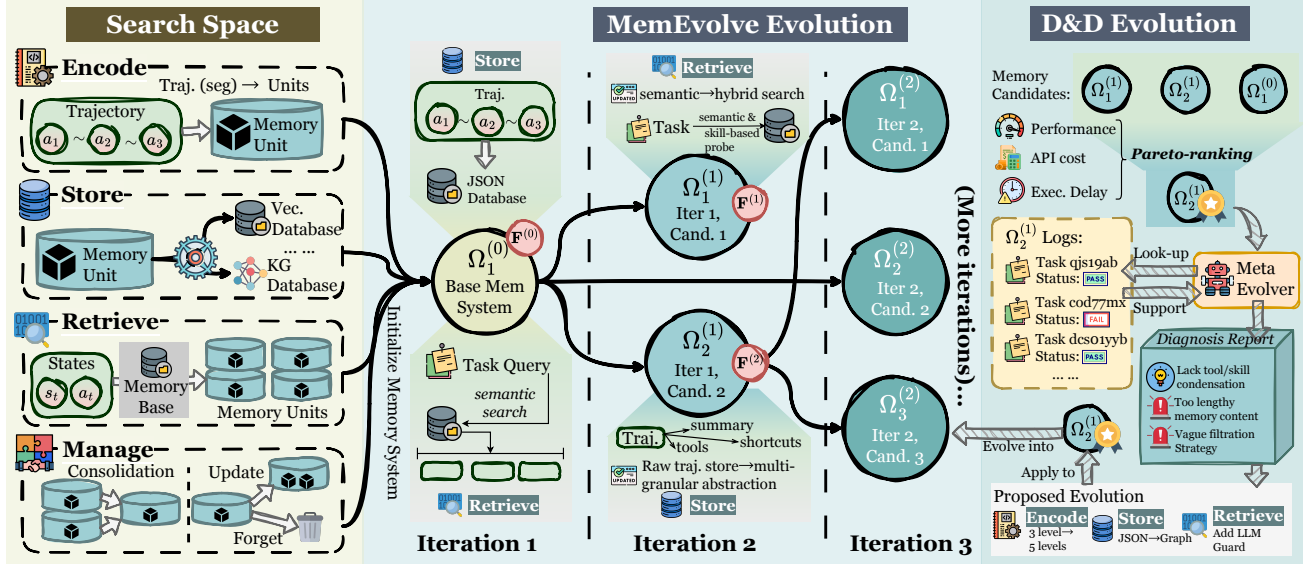


Figure 2. The overview meta-evolving process of our proposed MemEvolve.

empty initialization under a fixed set of architectures, and (ii) evolving the *memory architectures* themselves based on the induced performance:

$$\begin{aligned} &(\{\emptyset\}_{j \in \mathcal{J}^{(k)}}, \{\Omega_j^{(k)}\}_{j \in \mathcal{J}^{(k)}}) \xrightarrow{\text{inner}} (\{M_{t+1,j}^{(k)}\}_j, \{\Omega_j^{(k)}\}_j) \\ &\xrightarrow{\text{outer}} (\{M_{t+1,j}^{(k)}\}_j, \{\Omega_{j'}^{(k+1)}\}_{j' \in \mathcal{J}^{(k+1)}}). \end{aligned} \quad (6)$$

By iterating this dual-evolution process, the agent does not merely accumulate experience within a fixed memory system; instead, both the memory base and the governing memory architectures co-evolve, yielding increasingly adaptive and resource-aware memory-driven behavior over time.

4.2. Diagnose-and-Design Evolution

We now detail the meta-evolution operator $\mathcal{F}$, which governs the architectural update in each evolutionary iteration. Conceptually, $\mathcal{F}$ decomposes into two coordinated components: (i) *architectural selection*, which identifies a subset of high-performing memory systems to serve as evolutionary parents, and (ii) *diagnose-and-design evolution*, which generates new memory architectures from each selected parent through a structured diagnosis procedure followed by a constrained redesign within the modular space.

Architectural Selection. Given the candidate set $\{\Omega_j^{(k)}\}_{j \in \mathcal{J}^{(k)}}$ and their corresponding summaries $\{\mathbf{F}_j^{(k)}\}$, we define each summary vector as

$$\mathbf{F}_j^{(k)} \triangleq (\text{Perf}_j^{(k)}, -\text{Cost}_j^{(k)}, -\text{Delay}_j^{(k)}), \quad (7)$$

where higher values are preferred in all dimensions. Candidates are first ranked by non-dominated sorting over $\mathbf{F}_j^{(k)}$, yielding a Pareto rank $\rho_j^{(k)}$. Within the same Pareto rank, candidates are further ordered by performance $\text{Perf}_j^{(k)}$. The

top- K candidates are selected as the parent set:

$$\mathcal{P}^{(k)} = \text{Top-K} \left(\rho_j^{(k)}, \text{Perf}_j^{(k)} \right)_{j \in \mathcal{J}^{(k)}}. \quad (8)$$

This selection step ensures that architectural evolution is guided by systems that exhibit favorable trade-offs between task effectiveness and resource efficiency, while prioritizing task performance among Pareto-equivalent candidates.

Diagnose-and-Design Evolution. For each parent architecture $\Omega_p^{(k)} \in \mathcal{P}^{(k)}$, $\mathcal{F}$ generates a set of S descendants $\{\Omega_{p,s}^{(k+1)}\}_{s=1}^S$ through a two-phase process:

- **Diagnosis.** Each parent architecture is examined using trajectory-level evidence from its own execution batch $\mathcal{T}_p^{(k)}$. For each trajectory, the agent provides outcome statistics (e.g., success indicators, token costs) together with a structured description of the associated task query. A replay interface grants access to the corresponding trajectories $\tau \in \mathcal{T}_p^{(k)}$, enabling targeted inspection of memory behavior, including retrieval failures, ineffective abstractions, or storage inefficiencies. The diagnosis phase thus produces a structured defect profile $\mathcal{D}(\Omega_p^{(k)})$, characterizing architectural bottlenecks across the four memory components $(\mathcal{E}_p^{(k)}, \mathcal{U}_p^{(k)}, \mathcal{R}_p^{(k)}, \mathcal{G}_p^{(k)})$.
- **Design.** Conditioned on the defect profile $\mathcal{D}(\Omega_p^{(k)})$, a redesigned architecture is constructed by modifying only the permissible implementation sites within the modular interface, thereby ensuring compatibility and isolating architectural changes to the designated design space. The design step produces S variants by instantiating distinct

Table 2. Performance of various agent frameworks on the WebWalkerQA, xBench-DS, TaskCraft, and GAIA benchmarks.

Framework	Model Family	WebWalker QA	xBench -DS	Task Craft	GAIA			
					Avg.	Level 1	Level 2	Level 3
OWL Workforce (pass@3)	GPT-4o+o3-mini	57.64	55.0	58.33	60.61	81.14	58.14	26.92
OWL RP (pass@3)	GPT-4o+o3-mini	-	-	-	58.18	81.14	54.65	23.08
TapeAgents	Claude 3.7 etc.	-	-	-	55.76	71.70	53.49	30.77
AutoAgent	Claude 3.5 etc.	-	-	-	55.15	71.70	53.40	26.92
Smolagents 🤖	GPT-4.1	-	-	-	55.15	67.92	53.49	34.62
Smolagents 🤖	GPT-5-mini	58.82	51.0	64.00	55.75	69.81	54.65	30.77
Magnetic-1	OpenAI o1 etc.	-	-	-	46.06	56.60	46.51	23.08
Cognitive Kernel-Pro (pass@1)	Claude-3.7 etc.	60.64	56.0	66.00	60.00	79.25	56.98	30.77
Cognitive Kernel-Pro (pass@3)	Claude-3.7 etc.	-	-	-	75.15	84.91	73.26	61.54
OAgents	Claude-3.7 etc.	58.23	47.0	-	66.67	77.36	66.28	46.15
JoyAgents	Claude-4, o4-mini	-	-	-	75.2	86.8	77.9	42.3
Agent KB (pass@1)	GPT-4.1	60.59	48.0	61.67	61.21	79.25	58.14	34.62
Agent KB (pass@2)	GPT-4.1	68.82	58.0	72.67	67.27	83.02	67.44	34.62
Agent KB (pass@3)	GPT-4.1	73.53	68.0	75.33	73.94	84.91	73.26	53.85
Flash-Searcher ⚡ (pass@1)	GPT-5-mini	71.18	69.0	69.67	69.09	79.25	69.77	46.15
Flash-Searcher ⚡ (pass@1)	Kimi K2	52.35	66.0	58.00	52.12	58.49	52.33	34.62
Flash-Searcher ⚡ (pass@1)	DeepSeek V3.2	69.41	68.0	69.33	60.61	79.25	53.49	46.15
MemEvolve + 🤖 (pass@1)	GPT-5-mini	61.18	57.0	67.67	64.24	83.02	58.14	46.15
MemEvolve + 🤖 (pass@2)	GPT-5-mini	67.06	63.0	75.00	67.88	84.91	63.95	46.15
MemEvolve + 🤖 (pass@3)	GPT-5-mini	71.18	68.0	77.00	72.12	88.68	68.60	50.00
MemEvolve + ⚡ (pass@1)	GPT-5-mini	74.71	74.0	72.00	73.33	83.02	73.26	53.85
MemEvolve + ⚡ (pass@2)	GPT-5-mini	79.41	77.0	75.00	77.58	92.45	74.42	57.69
MemEvolve + ⚡ (pass@3)	GPT-5-mini	81.18	78.0	79.33	80.61	94.34	79.07	57.69
MemEvolve + ⚡ (pass@1)	Kimi K2	69.41	68.0	68.00	61.21	67.92	63.95	38.46
MemEvolve + ⚡ (pass@1)	DeepSeek V3.2	72.35	70.0	72.67	67.88	83.02	63.95	50.00

but valid configurations of the four components:

$$\Omega_{p,s}^{(k+1)} = \text{Design}\left(\Omega_p^{(k)}, \mathcal{D}(\Omega_p^{(k)}), s\right), s \in \{1, \dots, S\}. \quad (9)$$

These variants differ in encoding strategies, storage rules, retrieval constraints, or management policies, yet all conform to the unified memory-system interface and remain executable by the agent.

Resulting update. Aggregating all descendants across parents yields the next set of candidate architectures:

$$\{\Omega_{j'}^{(k+1)}\}_{j' \in \mathcal{J}^{(k+1)}} = \bigcup_{\Omega_p^{(k)} \in \mathcal{P}^{(k)}} \{\Omega_{p,s}^{(k+1)}\}_{s=1}^S. \quad (10)$$

This diagnose-and-design evolution operationalizes $\mathcal{F}$ for producing increasingly adaptive memory systems, ensuring that architectural updates are both empirically grounded and structurally constrained within the unified design space.

5. Experiments

5.1. Experiment Setup

Benchmarks We evaluate the proposed framework across four challenging agentic benchmarks, including *GAIA* (Mi-

alon et al., 2023), *WebWalkerQA* (Wu et al., 2025a), *xBench-DeepSearch* (xBench-DS) (Chen et al., 2025), as well as *TaskCraft* (Shi et al., 2025a). Further statistics and details are provided in Section B.1.

Method Configurations We run the dual-evolution process for $K_{\max} = 3$ iterations. In the outer loop, the survivor budget is set as $K = 1$; at each iteration, only the top-ranked architecture is retained and expanded to $S = 3$ descendants. In the inner loop, each candidate architecture $\Omega_j^{(k)}$ is evaluated on a batch $\mathcal{T}_j^{(k)}$ of 60 task trajectories, consisting of 40 newly sampled tasks and 20 tasks reused from the previous iteration to stabilize inter-iteration comparison.

Agent Framework We integrate **MemEvolve** into two representative agentic frameworks: 🤖 *SmolAgent* (Roucher et al., 2025), a lightweight two-agent architecture, and ⚡ *Flash-Searcher* (Qin et al., 2025), a high-performance single-agent deep research system. To assess the generalization and plug-and-play capability of **MemEvolve**, we further evaluate it on two held-out multi-agent systems: Tencent’s 🧠 *Cognitive Kernel-Pro (CK-Pro)* (Fang et al., 2025c), a three-agent framework comprising main/file/web agents; and 🦉 *OWL* (Hu et al., 2025b), a hierarchical system including planner, coordinator, web, document, and coding

Table 3. Performance, cost, delay, and steps across datasets under different memory settings for ⚡ Flash-Searcher. Here, *cost* denotes the average API cost incurred per task query, *delay* measures the average execution latency (seconds) per task, and *#steps* reports the number of agent interaction steps required to complete each task.

Memory Setting	GAIA				xBench				WebWalkerQA			
	Perf.	Cost	Delay	#Steps	Perf.	Cost	Delay	#Steps	Perf.	Cost	Delay	#Steps
No-Memory	69.09	0.086	505.46	10.44	69.00	0.141	523.05	14.69	71.18	0.048	251.57	6.91
Generative	66.67	0.061	436.26	8.87	70.00	0.131	818.37	13.45	72.35	0.045	268.56	6.64
Voyager	69.70	0.060	499.89	9.25	68.00	0.117	553.46	12.71	73.53	0.049	333.69	6.99
DILU	66.67	0.059	444.62	8.91	69.00	0.134	500.72	13.83	72.94	0.046	272.16	6.96
ExpeL	66.06	0.059	500.11	8.68	64.00	0.123	710.32	13.05	69.41	0.076	385.28	10.96
AWM	67.27	0.062	584.88	10.23	71.00	0.138	761.33	14.12	72.35	0.068	397.20	11.40
Mobile-E	69.09	0.065	321.80	9.35	68.00	0.120	537.18	13.16	71.76	0.059	296.01	6.52
Cheatsheet	68.48	0.069	559.81	9.72	65.00	0.174	818.07	15.99	72.94	0.057	367.13	7.59
MemEvolve	73.33	0.085	693.33	10.14	74.00	0.136	773.06	14.20	74.71	0.040	332.49	6.64

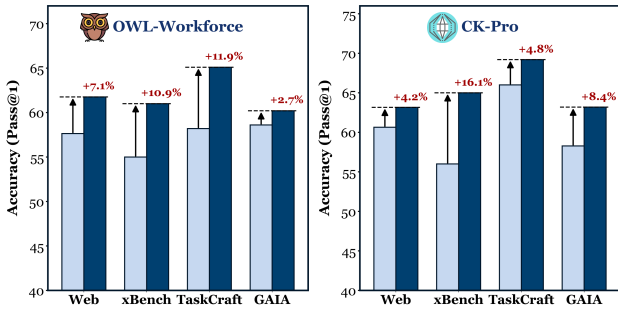


Figure 3. The cross-framework generalization analysis. We transfer the memory system evolved on TaskCraft+⚡ to 🦉 and 🧠. Red percentages denote the relative score gains of each framework after integrating MemEvolve over its memory-free counterpart.

agents. This diversity in architecture and system complexity enables a comprehensive examination of the adaptability of MemEvolve across heterogeneous agentic scaffolds.

Model Configurations We instantiate MemEvolve using GPT-5-MINI (OpenAI, 2025) as the LLM backbone for the underlying agentic frameworks, and for supporting the meta-evolution operator $\mathcal{F}(\cdot)$. To further evaluate the cross-LLM generalization capability of MemEvolve, we additionally consider alternative backbones, including DEEPSEEK V3.2 (DeepSeek-AI et al., 2025), and KIMI K2 (Team et al., 2025a). We explicitly report the specific LLM backbone used by each framework in the following experiments.

5.2. Main Results

We report the pass@1–3 performance of MemEvolve integrated with SmolAgent and Flash-Searcher in Table 2, together with its generalization results when paired with unseen LLMs (KIMI K2, DEEPSEEK V3.2). Notably, on the relatively simple TaskCraft, we evolve two distinct memory systems using MemEvolve+🧠 and MemEvolve+⚡, respectively. These evolved memory systems are then fixed and evaluated on WebWalkerQA and xBench-DS, *i.e.*, without conducting dataset-specific meta-evolution.

MemEvolve Exhibits Cross-Task, Cross-Model, and Cross-Framework Generalization.

As shown in Table 2, memory architecture is a primary determinant of agent performance and transferability. On xBench, 🧠+GPT-5-MINI attains a pass@1 of 51%, which increases by 6% and reaches 68.0% at pass@3 after integrating MemEvolve; similarly, ⚡+GPT-5-MINI improves from 69% to 74%. In contrast to prior approaches that rely on task-specific tuning or static memory pipelines, the memories evolved by MemEvolve transfer without dataset-level meta-evolution: memory systems learned on TaskCraft yield consistent gains on harder benchmarks (WebWalkerQA+🧠: 58.82 → 61.18%; xBench+⚡: 69.0 → 74.0%), indicating task-agnostic design principles rather than dataset overfitting. Beyond cross-task transfer, MemEvolve exhibits strong cross-LLM generalization: although meta-evolution is conducted with GPT-5-MINI, the evolved memories transfer effectively to KIMI K2 and DEEPSEEK V3.2 without manual adaptation, with KIMI K2+⚡ achieving gains of 17.06% on WebWalkerQA and 10.0% on TaskCraft. Finally, as shown in Figure 3, directly plugging these memories into heterogeneous agentic frameworks (*e.g.*, 🦉, 🧠) consistently improves performance despite substantial architectural differences, underscoring that MemEvolve learns framework-agnostic, reusable memory abstractions rather than narrowly task-bound heuristics.

5.3. Self-Evolving Memory Comparison

We further compare the memory systems automatically evolved by MemEvolve against prevailing human-designed self-improving memory systems. In Table 3, we integrate seven representative self-improving memory systems implemented in EvolveLab with Flash-Searcher, and comprehensively report performance, per-task cost/execution latency/execution steps. Results for MemEvolve are obtained using the system evolved on TaskCraft+⚡+GPT-5-MINI.

MemEvolve Delivers Robust and Consistent Improvements. Despite faithful re-implementations, many exist-

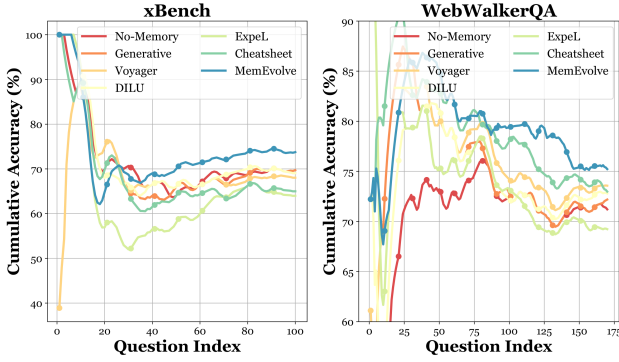


Figure 4. Evolution of cumulative accuracy across question indices. Cumulative accuracy at index i is defined as the average accuracy over the first i questions. The curves exhibit larger fluctuations at early indices due to limited sample size, and gradually stabilize as more questions are accumulated.

ing memory systems fail to deliver consistent improvements across benchmarks. For instance, DILU improves xBench and WebWalkerQA yet degrades GAIA by 2.42%, while Dynamic Cheatsheet yields a modest 1.76% gain on WebWalkerQA but performs poorly on GAIA and xBench. ExpeL underperforms across all three, reflecting its original design for relatively simple embodied or QA settings (e.g., ALFWorld, HotpotQA) and its limited suitability for long-horizon, long-context deep research. In contrast, **MemEvolve** produces stable, benchmark-agnostic gains: although evolved on TaskCraft, it consistently improves performance by 3.54%~5.0% across all evaluated benchmarks. These improvements are achieved without substantial cost inflation, maintaining API expenses comparable to the No-Memory baseline (e.g., GAIA: \$0.085 vs. \$0.086; xBench: \$0.136 vs. \$0.141) and execution delays on par with other self-improving methods (e.g., GAIA: 693.33s vs. 584.88s for AWM and 559.81s for Cheatsheet; xBench: 773.06s vs. 761.33s for AWM and 818.07s for Cheatsheet). As shown in Figure 4, while early-stage performance exhibits higher variance, **MemEvolve** converges to a consistently superior regime, indicating principled, transferable memory designs rather than brittle, task-specific heuristics. Three-run evaluations in Table 5 yield consistent rankings under repeated trials.

5.4. Framework Analysis

Ablation Study We isolate the contribution of each memory module by evolving only one component at a time while freezing the other three components at the R0 AgentKB initialization. As shown in Table 4, evolving retrieval alone yields the largest single-module gain (+10.00), consistent with the qualitative observation that early evolution shifts retrieval from static semantic matching toward agent-gated, context-aware filtering. Nevertheless, no single component matches the full framework: joint evolution improves

Table 4. Component-level ablation on TaskCraft. Each “Only” variant evolves one memory module while keeping the remaining modules fixed at the R0 AgentKB initialization; Full **MemEvolve** jointly evolves all modules.

Round	Only Enc.	Only Ret.	Only Mng.	Only Store	Full
R0	61.67	61.67	61.67	61.67	61.67
R1 best	63.33	66.67	63.33	63.33	70.00
R2 best	66.67	71.67	65.00	63.33	75.00
R3 best	68.33	71.67	66.67	65.00	78.33
Δ	+6.66	+10.00	+5.00	+3.33	+16.66

TaskCraft by +16.66, indicating that the final gains arise from cross-module coupling, such as improved encoders producing memory entries that are more effectively exploited by adaptive retrieval and management.

Sensitivity Analysis The default evolutionary budget ($K = 1, S = 3$) follows a compact $(1 + \lambda)$ -style strategy: one survivor seeds the next generation, while three descendants provide limited but structured exploration under expensive agent-level fitness evaluation. We further examine sensitivity to the evolution horizon by extending the process to five rounds in Table 6. Performance continues to improve from R3 to R5 on both in-domain TaskCraft (78.33 $\rightarrow$ 80.00) and OOD WebWalkerQA (76.27 $\rightarrow$ 77.65), while cost and latency decrease further. This supports $K_{\max} = 3$ as an efficient default while suggesting that longer evolution can further optimize resource efficiency. Cross-backbone and search-baseline analyses are provided in Tables 7 and 8.

5.5. Meta-Evolving Dynamics

Having established the performance gains of **MemEvolve**, we examine how meta-evolution proceeds in practice and which components are modified. As shown in Figure 6, **MemEvolve** initializes from the predefined AgentKB structure and iteratively evolves toward more efficient memory architectures. Figures 8 and 9 highlight two high-performing systems discovered along this trajectory, RIVA and CEREBRA, while Figure 7 presents a system evolved from a minimal few-shot baseline, denoted LIGHTWEIGHT.

Agents Spontaneously Evolve Efficient Memory Architectures Starting from AgentKB with frozen encoding and storage, **MemEvolve** explores a range of evolutionary directions, from aggressive designs (e.g., $\Omega_1^{(1)}$, which decomposes trajectories into 9 fine-grained skills) to conservative ones (e.g., $\Omega_3^{(1)}$, which stores trajectories at four levels and applies an LLM guardrail during retrieval), with the latter winning in the first round. This stage is marked by increasing *agentic* control, as encoding and retrieval shift from predefined pipelines to agent-driven decisions. By the third round, the transition from RIVA to CEREBRA introduces two advances: distillation of reusable tools/insights and periodic database maintenance, yielding faster and more stable evolutionary progress for downstream agents.

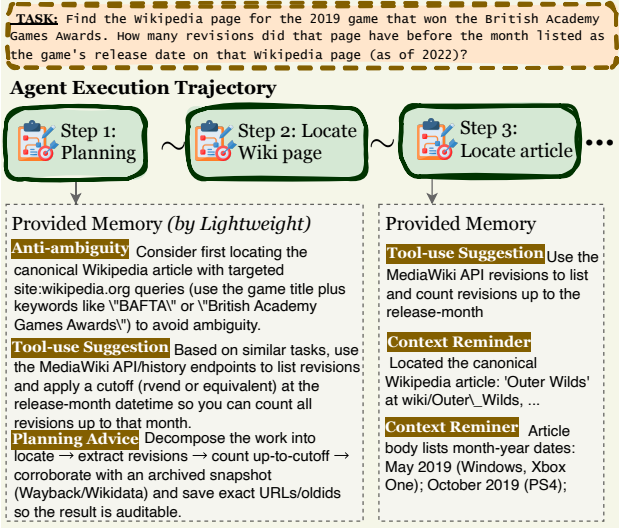


Figure 5. Illustration of how evolved memories are instantiated. The memory system adaptively provides stage-specific guidance, ranging from high-level planning and task decomposition to fine-grained tool-use suggestions and salient context recall.

Evolved Memory Systems Are Effective in Practice Qualitative examples from LIGHTWEIGHT (Figure 5) demonstrate adaptive memory behavior across task phases. During planning, the system provides high-level guidance for task decomposition; during execution, it supplies fine-grained tool-use recommendations and maintains a working memory that highlights salient prior information. These illustrate the practical utility of the evolved memory systems.

6. Conclusion

This work introduces a unified design space and standardized codebase, **EvolveLab**, for self-evolving agent memory, upon which we build **MemEvolve**, a meta-evolutionary memory framework. Departing from manually crafting a single memory architecture and assuming cross-domain generalization, **MemEvolve** enables adaptive, memory architecture evolution in bi-level optimization. Extensive evaluations across diverse benchmarks and LLMs demonstrate its effectiveness, robustness, and generalization.

Impact Statement

Ethical Considerations. This work studies meta-evolving memory architectures for LLM-based agents and evaluates the proposed framework exclusively on publicly available benchmarks and synthetic or controlled interaction trajectories. The method does not involve the collection of personal data, nor does it require access to sensitive, private, or proprietary information. We therefore do not identify specific ethical risks beyond the well-known considerations associated with deploying large language models, such as potential misuse or overreliance on automated systems.

Societal Impact. By enabling memory systems that adapt to task demands and transfer across models and frameworks, this work aims to improve the robustness, efficiency, and generalization of agentic systems in long-horizon and complex settings. More reliable and transparent agent behavior may benefit applications in research assistance and software development. We expect the primary societal impact to be positive, facilitating more responsible design, evaluation, and deployment of self-improving AI systems.

Acknowledgements

This work was supported by the National Natural Science Foundation of China under Grant No. 62320106007, and by NUS Grant A-0010106-00-00, A-8004365-00-00 and A-8004410-01-00.

References

- Bai, L., Cai, Z., Cao, Y., Cao, M., Cao, W., Chen, C., Chen, H., Chen, K., Chen, P., Chen, Y., Chen, Y., Cheng, Y., Chu, P., Chu, T., Cui, E., Cui, G., Cui, L., Cui, Z., Deng, N., Ding, N., Dong, N., Dong, P., Dou, S., Du, S., Duan, H., Fan, C., Gao, B., Gao, C., Gao, J., Gao, S., Gao, Y., Gao, Z., Ge, J., Ge, Q., Gu, L., Gu, Y., Guo, A., Guo, Q., Guo, X., He, C., He, J., Hong, Y., Hou, S., Hu, C., Hu, H., Hu, J., Hu, M., Hua, Z., Huang, H., Huang, J., Huang, X., Huang, Z., Jiang, Z., Kong, L., Li, L., Li, P., Li, P., Li, S., Li, T., Li, W., Li, Y., Lin, D., Lin, J., Lin, T., Lin, Z., Liu, H., Liu, J., Liu, J., Liu, J., Liu, K., Liu, K., Liu, K., Liu, S., Liu, S., Liu, W., Liu, X., Liu, Y., Liu, Z., Lu, Y., Lv, H., Lv, H., Lv, H., Lv, Q., Lv, Y., Lyu, C., Ma, C., Ma, J., Ma, R., Ma, R., Ma, R., Ma, X., Ma, Y., Ma, Z., Mi, S., Ning, J., Ning, W., Pang, X., Peng, J., Peng, R., Qiao, Y., Qiu, J., Qu, X., Qu, Y., Ren, Y., Shang, F., Shao, W., Shen, J., Shen, S., Song, C., Song, D., Song, D., Su, C., Su, W., Sun, W., Sun, Y., Tan, Q., Tang, C., Tang, H., Tang, K., Tang, S., Tong, J., Wang, A., Wang, B., Wang, D., Wang, L., Wang, R., Wang, W., Wang, W., Wang, J., Wang, Y., Wang, Z., Wu, L.-I., Wu, W., Wu, Y., Wu, Z., Xiao, L., Xing, S., Xu, C., Xu, H., Xu, J., Xu, R., Xu, W., Yang, G., Yang, Y., Ye, H., Ye, J., Ye, S., Yu, J., Yu, J., Yu, J., Yuan, F., Zang, Y., Zhang, B., Zhang, C., Zhang, C., Zhang, H., Zhang, J., Zhang, Q., Zhang, Q., Zhang, S., Zhang, T., Zhang, W., Zhang, W., Zhang, Y., Zhang, Z., Zhao, H., Zhao, Q., Zhao, X., Zhao, X., Zhou, B., Zhou, D., Zhou, P., Zhou, Y., Zhou, Y., Zhu, D., Zhu, L., and Zou, Y. Intern-s1: A scientific multimodal foundation model, 2025. URL <https://arxiv.org/abs/2508.15763>.
- Cai, Y., Cai, S., Shi, Y., Xu, Z., Chen, L., Qin, Y., Tan, X., Li, G., Li, Z., Lin, H., Mao, Y., Li, K., and Sun, X. Training-free group relative policy optimization, 2025. URL <https://arxiv.org/abs/2510.08191>.

- Chen, K., Ren, Y., Liu, Y., Hu, X., Tian, H., Xie, T., Liu, F., Zhang, H., Liu, H., Gong, Y., Sun, C., Hou, H., Yang, H., Pan, J., Lou, J., Mao, J., Liu, J., Li, J., Liu, K., Liu, K., Wang, R., Li, R., Niu, T., Zhang, W., Yan, W., Wang, X., Zhang, Y., Hung, Y.-H., Jiang, Y., Liu, Z., Yin, Z., Ma, Z., and Mo, Z. xbench: Tracking agents productivity scaling with profession-aligned real-world evaluations, 2025. URL <https://arxiv.org/abs/2506.13651>.
- DeepSeek-AI, Liu, A., Mei, A., Lin, B., Xue, B., Wang, B., Xu, B., Wu, B., Zhang, B., Lin, C., Dong, C., Lu, C., Zhao, C., Deng, C., Xu, C., Ruan, C., Dai, D., Guo, D., Yang, D., Chen, D., Li, E., Zhou, F., Lin, F., Dai, F., Hao, G., Chen, G., Li, G., Zhang, H., Xu, H., Li, H., Liang, H., Wei, H., Zhang, H., Luo, H., Ji, H., Ding, H., Tang, H., Cao, H., Gao, H., Qu, H., Zeng, H., Huang, J., Li, J., Xu, J., Hu, J., Chen, J., Xiang, J., Yuan, J., Cheng, J., Zhu, J., Ran, J., Jiang, J., Qiu, J., Li, J., Song, J., Dong, K., Gao, K., Guan, K., Huang, K., Zhou, K., Huang, K., Yu, K., Wang, L., Zhang, L., Wang, L., Zhao, L., Yin, L., Guo, L., Luo, L., Ma, L., Wang, L., Zhang, L., Di, M. S., Xu, M. Y., Zhang, M., Zhang, M., Tang, M., Zhou, M., Huang, P., Cong, P., Wang, P., Wang, Q., Zhu, Q., Li, Q., Chen, Q., Du, Q., Xu, R., Ge, R., Zhang, R., Pan, R., Wang, R., Yin, R., Xu, R., Shen, R., Zhang, R., Liu, S. H., Lu, S., Zhou, S., Chen, S., Cai, S., Chen, S., Hu, S., Liu, S., Hu, S., Ma, S., Wang, S., Yu, S., Zhou, S., Pan, S., Zhou, S., Ni, T., Yun, T., Pei, T., Ye, T., Yue, T., Zeng, W., Liu, W., Liang, W., Pang, W., Luo, W., Gao, W., Zhang, W., Gao, X., Wang, X., Bi, X., Liu, X., Wang, X., Chen, X., Zhang, X., Nie, X., Cheng, X., Liu, X., Xie, X., Liu, X., Yu, X., Li, X., Yang, X., Li, X., Chen, X., Su, X., Pan, X., Lin, X., Fu, X., Wang, Y. Q., Zhang, Y., Xu, Y., Ma, Y., Li, Y., Li, Y., Zhao, Y., Sun, Y., Wang, Y., Qian, Y., Yu, Y., Zhang, Y., Ding, Y., Shi, Y., Xiong, Y., He, Y., Zhou, Y., Zhong, Y., Piao, Y., Wang, Y., Chen, Y., Tan, Y., Wei, Y., Ma, Y., Liu, Y., Yang, Y., Guo, Y., Wu, Y., Wu, Y., Cheng, Y., Ou, Y., Xu, Y., Wang, Y., Gong, Y., Wu, Y., Zou, Y., Li, Y., Xiong, Y., Luo, Y., You, Y., Liu, Y., Zhou, Y., Wu, Z. F., Ren, Z. Z., Zhao, Z., Ren, Z., Sha, Z., Fu, Z., Xu, Z., Xie, Z., Zhang, Z., Hao, Z., Gou, Z., Ma, Z., Yan, Z., Shao, Z., Huang, Z., Wu, Z., Li, Z., Zhang, Z., Xu, Z., Wang, Z., Gu, Z., Zhu, Z., Li, Z., Zhang, Z., Xie, Z., Gao, Z., Pan, Z., Yao, Z., Feng, B., Li, H., Cai, J. L., Ni, J., Xu, L., Li, M., Tian, N., Chen, R. J., Jin, R. L., Li, S. S., Zhou, S., Sun, T., Li, X. Q., Jin, X., Shen, X., Chen, X., Song, X., Zhou, X., Zhu, Y. X., Huang, Y., Li, Y., Zheng, Y., Zhu, Y., Ma, Y., Huang, Z., Xu, Z., Zhang, Z., Ji, D., Liang, J., Guo, J., Chen, J., Xia, L., Wang, M., Li, M., Zhang, P., Chen, R., Sun, S., Wu, S., Ye, S., Wang, T., Xiao, W. L., An, W., Wang, X., Sun, X., Wang, X., Tang, Y., Zha, Y., Zhang, Z., Ju, Z., Zhang, Z., and Qu, Z. Deepseek-v3.2: Pushing the frontier of open large language models, 2025. URL <https://arxiv.org/abs/2512.02556>.
- Du, M., Xu, B., Zhu, C., Wang, X., and Mao, Z. Deep-research bench: A comprehensive benchmark for deep research agents, 2025. URL <https://arxiv.org/abs/2506.11763>.
- Fang, J., Peng, Y., Zhang, X., Wang, Y., Yi, X., Zhang, G., Xu, Y., Wu, B., Liu, S., Li, Z., Ren, Z., Aletras, N., Wang, X., Zhou, H., and Meng, Z. A comprehensive survey of self-evolving ai agents: A new paradigm bridging foundation models and lifelong agentic systems, 2025a. URL <https://arxiv.org/abs/2508.07407>.
- Fang, R., Liang, Y., Wang, X., Wu, J., Qiao, S., Xie, P., Huang, F., Chen, H., and Zhang, N. Memp: Exploring agent procedural memory, 2025b. URL <https://arxiv.org/abs/2508.06433>.
- Fang, T., Zhang, Z., Wang, X., Wang, R., Qin, C., Wan, Y., Ma, J.-Y., Zhang, C., Chen, J., Li, X., Zhang, H., Mi, H., and Yu, D. Cognitive kernel-pro: A framework for deep research agents and agent foundation models training, 2025c. URL <https://arxiv.org/abs/2508.00414>.
- Ghareeb, A. E., Chang, B., Mitchener, L., Yiu, A., Szostkiewicz, C. J., Laurent, J. M., Razzak, M. T., White, A. D., Hinks, M. M., and Rodrigues, S. G. Robin: A multi-agent system for automating scientific discovery, 2025. URL <https://arxiv.org/abs/2505.13400>.
- Hong, S., Zhuge, M., Chen, J., Zheng, X., Cheng, Y., Wang, J., Zhang, C., Wang, Z., Yau, S. K. S., Lin, Z., Zhou, L., Ran, C., Xiao, L., Wu, C., and Schmidhuber, J. MetaGPT: Meta programming for a multi-agent collaborative framework. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=VtmBAGCN7o>.
- Hu, M., Zhou, Y., Fan, W., Nie, Y., Xia, B., Sun, T., Ye, Z., Jin, Z., Li, Y., Chen, Q., Zhang, Z., Wang, Y., Ye, Q., Ghanem, B., Luo, P., and Li, G. Owl: Optimized workforce learning for general multi-agent assistance in real-world task automation, 2025a. URL <https://arxiv.org/abs/2505.23885>.
- Hu, M., Zhou, Y., Fan, W., Nie, Y., Xia, B., Sun, T., Ye, Z., Jin, Z., Li, Y., Zhang, Z., Wang, Y., Ye, Q., Luo, P., and Li, G. Owl: Optimized workforce learning for general multi-agent assistance in real-world task automation, 2025b. URL <https://github.com/camel-ai/owl>.
- Hu, Y., Liu, S., Yue, Y., Zhang, G., Liu, B., Zhu, F., Lin, J., Guo, H., Dou, S., Xi, Z., Jin, S., Tan, J., Yin, Y., Liu, J., Zhang, Z., Sun, Z., Zhu, Y., Sun, H., Peng, B., Cheng, Z., Fan, X., Guo, J., Yu, X., Zhou, Z., Hu, Z., Huo, J.,

- Wang, J., Niu, Y., Wang, Y., Yin, Z., Hu, X., Liao, Y., Li, Q., Wang, K., Zhou, W., Liu, Y., Cheng, D., Zhang, Q., Gui, T., Pan, S., Zhang, Y., Torr, P., Dou, Z., Wen, J.-R., Huang, X., Jiang, Y.-G., and Yan, S. Memory in the age of ai agents, 2025c. URL <https://arxiv.org/abs/2512.13564>.
- LangChain. Langchain: Build context-aware reasoning applications. [Online], 2023. <https://github.com/langchain-ai/langchain>.
- Mialon, G., Fourrier, C., Wolf, T., LeCun, Y., and Scialom, T. Gaia: a benchmark for general ai assistants. In *The Twelfth International Conference on Learning Representations*, 2023.
- OpenAI. Introducing GPT-5 — openai.com. <https://openai.com/index/introducing-gpt-5/>, 2025. [Accessed 16-12-2025].
- Ouyang, S., Yan, J., Hsu, I.-H., Chen, Y., Jiang, K., Wang, Z., Han, R., Le, L. T., Daruki, S., Tang, X., Tirumalashetty, V., Lee, G., Rofouei, M., Lin, H., Han, J., Lee, C.-Y., and Pfister, T. Reasoningbank: Scaling agent self-evolving with reasoning memory, 2025. URL <https://arxiv.org/abs/2509.25140>.
- Packer, C., Fang, V., Patil, S., Lin, K., Wooders, S., and Gonzalez, J. Memgpt: Towards llms as operating systems. 2023.
- Phan, L., Gatti, A., Han, Z., Li, N., Hu, J., Zhang, H., Zhang, C. B. C., Shaaban, M., Ling, J., Shi, S., et al. Humanity’s last exam. *arXiv preprint arXiv:2501.14249*, 2025.
- Qin, T., Chen, Q., Wang, S., Xing, H., Zhu, K., Zhu, H., Shi, D., Liu, X., Zhang, G., Liu, J., Jiang, Y. E., Gao, X., and Zhou, W. Flash-searcher: Fast and effective web agents via dag-based parallel execution, 2025. URL <https://arxiv.org/abs/2509.25301>.
- Qiu, J., Juan, X., Wang, Y., Yang, L., Qi, X., Zhang, T., Guo, J., Lu, Y., Yao, Z., Wang, H., Liu, S., Jiang, X., Leqi, L., and Wang, M. Agentdistill: Training-free agent distillation with generalizable mcp boxes, 2025a. URL <https://arxiv.org/abs/2506.14728>.
- Qiu, J., Qi, X., Zhang, T., Juan, X., Guo, J., Lu, Y., Wang, Y., Yao, Z., Ren, Q., Jiang, X., et al. Alita: Generalist agent enabling scalable agentic reasoning with minimal predefinition and maximal self-evolution. *arXiv preprint arXiv:2505.20286*, 2025b.
- Rasmussen, P., Paliychuk, P., Beauvais, T., Ryan, J., and Chalef, D. Zep: A temporal knowledge graph architecture for agent memory, 2025. URL <https://arxiv.org/abs/2501.13956>.
- Roucher, A., del Moral, A. V., Wolf, T., von Werra, L., and Kaunismäki, E. ‘smolagents’: a smol library to build great agentic systems. <https://github.com/huggingface/smolagents>, 2025.
- Shi, D., Cao, J., Chen, Q., Sun, W., Li, W., Lu, H., Dong, F., Qin, T., Zhu, K., Yang, M., Yang, J., Zhang, G., Liu, J., Zhang, C., Wang, J., Jiang, Y. E., and Zhou, W. Taskcraft: Automated generation of agentic tasks, 2025a. URL <https://arxiv.org/abs/2506.10055>.
- Shi, Y., Wang, M., Cao, Y., Lai, H., Lan, J., Han, X., Wang, Y., Geng, J., Li, Z., Xia, Z., et al. Aime: Towards fully-autonomous multi-agent framework. *arXiv preprint arXiv:2507.11988*, 2025b.
- Significant-Gravitas. Autogpt. [Online], 2023. <https://github.com/Significant-Gravitas/AutoGPT>.
- Sun, H. and Zeng, S. Hierarchical memory for high-efficiency long-term reasoning in llm agents, 2025. URL <https://arxiv.org/abs/2507.22925>.
- Suzgun, M., Yuksekogonul, M., Bianchi, F., Jurafsky, D., and Zou, J. Dynamic cheatsheet: Test-time learning with adaptive memory, 2025. URL <https://arxiv.org/abs/2504.07952>.
- Tang, X., Hu, T., Ye, M., Shao, Y., Yin, X., Ouyang, S., Zhou, W., Lu, P., Zhang, Z., Zhao, Y., Cohan, A., and Gerstein, M. Chemagent: Self-updating library in large language models improves chemical reasoning, 2025. URL <https://arxiv.org/abs/2501.06590>.
- Team, K., Bai, Y., Bao, Y., Chen, G., Chen, J., Chen, N., Chen, R., Chen, Y., Chen, Y., Chen, Y., Chen, Z., Cui, J., Ding, H., Dong, M., Du, A., Du, C., Du, D., Du, Y., Fan, Y., Feng, Y., Fu, K., Gao, B., Gao, H., Gao, P., Gao, T., Gu, X., Guan, L., Guo, H., Guo, J., Hu, H., Hao, X., He, T., He, W., He, W., Hong, C., Hu, Y., Hu, Z., Huang, W., Huang, Z., Huang, Z., Jiang, T., Jiang, Z., Jin, X., Kang, Y., Lai, G., Li, C., Li, F., Li, H., Li, M., Li, W., Li, Y., Li, Y., Li, Z., Li, Z., Lin, H., Lin, X., Lin, Z., Liu, C., Liu, C., Liu, H., Liu, J., Liu, J., Liu, L., Liu, S., Liu, T. Y., Liu, T., Liu, W., Liu, Y., Liu, Y., Liu, Y., Liu, Y., Liu, Z., Lu, E., Lu, L., Ma, S., Ma, X., Ma, Y., Mao, S., Mei, J., Men, X., Miao, Y., Pan, S., Peng, Y., Qin, R., Qu, B., Shang, Z., Shi, L., Shi, S., Song, F., Su, J., Su, Z., Sun, X., Sung, F., Tang, H., Tao, J., Teng, Q., Wang, C., Wang, D., Wang, F., Wang, H., Wang, J., Wang, J., Wang, J., Wang, S., Wang, S., Wang, Y., Wang, Y., Wang, Y., Wang, Y., Wang, Y., Wang, Z., Wang, Z., Wang, Z., Wei, C., Wei, Q., Wu, W., Wu, X., Wu, Y., Xiao, C., Xie, X., Xiong, W., Xu, B., Xu, J., Xu, J., Xu, L. H., Xu, L., Xu, S., Xu, W., Xu, X., Xu, Y., Xu, Z., Yan, J., Yan, Y., Yang, X., Yang, Y., Yang, Z., Yang, Z., Yang, Z., Yao, H., Yao, X., Ye, W., Ye,

- Z., Yin, B., Yu, L., Yuan, E., Yuan, H., Yuan, M., Zhan, H., Zhang, D., Zhang, H., Zhang, W., Zhang, X., Zhang, Y., Zhang, Y., Zhang, Y., Zhang, Y., Zhang, Y., Zhang, Y., Zhang, Z., Zhao, H., Zhao, Y., Zheng, H., Zheng, S., Zhou, J., Zhou, X., Zhou, Z., Zhu, Z., Zhuang, W., and Zu, X. Kimi k2: Open agentic intelligence, 2025a. URL <https://arxiv.org/abs/2507.20534>.
- Team, M. L., Bayan, Li, B., Lei, B., Wang, B., Rong, B., Wang, C., Zhang, C., Gao, C., Zhang, C., Sun, C., Han, C., Xi, C., Zhang, C., Peng, C., Qin, C., Zhang, C., Chen, C., Wang, C., Ma, D., Pan, D., Bu, D., Zhao, D., Kong, D., Liu, D., Huo, F., Li, F., Zhang, F., Dong, G., Liu, G., Xu, G., Li, G., Tan, G., Lin, G., Jing, H., Fu, H., Yan, H., Wen, H., Zhao, H., Liu, H., Shi, H., Hao, H., Tang, H., Lv, H., Su, H., Li, J., Liu, J., Li, J., Yang, J., Wang, J., Yang, J., Tan, J., Sun, J., Zhang, J., Fu, J., Yang, J., Hu, J., Qin, J., Wang, J., He, J., Kuang, J., Mei, J., Liang, K., He, K., Zhang, K., Wang, K., He, K., Gao, L., Shi, L., Ma, L., Qiu, L., Kong, L., Si, L., Lyu, L., Guo, L., Yang, L., Yan, L., Xia, M., Gao, M., Zhang, M., Zhou, M., Shen, M., Tuo, M., Zhu, M., Li, P., Pei, P., Zhao, P., Jia, P., Sun, P., Gu, Q., Li, Q., Li, Q., Huang, Q., Duan, Q., Meng, R., Weng, R., Shao, R., Li, R., Wu, S., Liang, S., Wang, S., Dang, S., Fang, T., Li, T., Chen, T., Bai, T., Zhou, T., Xie, T., He, W., Huang, W., Liu, W., Shi, W., Wang, W., Wu, W., Zhao, W., Zan, W., Shi, W., Nan, X., Su, X., Li, X., Mei, X., Ji, X., Xi, X., Huang, X., Li, X., Fu, X., Liu, X., Wei, X., Cai, X., Chen, X., Liu, X., Li, X., Shi, X., Li, X., Wang, X., Chen, X., Hu, X., Miao, X., He, X., Zhang, X., Hao, X., Cao, X., Cai, X., Yang, X., Feng, Y., Bai, Y., Chen, Y., Yang, Y., Huo, Y., Sun, Y., Lu, Y., Zhang, Y., Zang, Y., Zhai, Y., Li, Y., Yin, Y., Lv, Y., Zhou, Y., Yang, Y., Xie, Y., Sun, Y., Zheng, Y., Wei, Y., Qian, Y., Liang, Y., Tai, Y., Zhao, Y., Yu, Z., Zhang, Z., Yang, Z., Zhang, Z., Xia, Z., Zou, Z., Zeng, Z., Su, Z., Chen, Z., Zhang, Z., Wang, Z., Jiang, Z., Zhao, Z., Wang, Z., and Su, Z. Longcat-flash technical report, 2025b. URL <https://arxiv.org/abs/2509.01322>.
- Tran, K.-T., Dao, D., Nguyen, M.-D., Pham, Q.-V., O’Sullivan, B., and Nguyen, H. D. Multi-agent collaboration mechanisms: A survey of llms, 2025. URL <https://arxiv.org/abs/2501.06322>.
- Wang, G., Xie, Y., Jiang, Y., Mandlkar, A., Xiao, C., Zhu, Y., Fan, L., and Anandkumar, A. Voyager: An Open-Ended Embodied Agent with Large Language Models. *arXiv e-prints*, art. arXiv:2305.16291, May 2023.
- Wang, W., Piękos, P., Nanbo, L., Laakom, F., Chen, Y., Ostaszewski, M., Zhuge, M., and Schmidhuber, J. Huxley-gödel machine: Human-level coding agent development by an approximation of the optimal self-improving machine, 2025a. URL <https://arxiv.org/abs/2510.21614>.
- Wang, X., Li, B., Song, Y., Xu, F. F., Tang, X., Zhuge, M., Pan, J., Song, Y., Li, B., Singh, J., Tran, H. H., Li, F., Ma, R., Zheng, M., Qian, B., Shao, Y., Muennighoff, N., Zhang, Y., Hui, B., Lin, J., Brennan, R., Peng, H., Ji, H., and Neubig, G. OpenHands: An Open Platform for AI Software Developers as Generalist Agents, 2024a. URL <https://arxiv.org/abs/2407.16741>.
- Wang, Y., Yang, L., Li, G., Wang, M., and Aragam, B. Scoreflow: Mastering llm agent workflows via score-based preference optimization, 2025b. URL <https://arxiv.org/abs/2502.04306>.
- Wang, Z., Xu, H., Wang, J., Zhang, X., Yan, M., Zhang, J., Huang, F., and Ji, H. Mobile-agent-e: Self-evolving mobile assistant for complex tasks, 2025c. URL <https://arxiv.org/abs/2501.11733>.
- Wang, Z. Z., Mao, J., Fried, D., and Neubig, G. Agent workflow memory, 2024b. URL <https://arxiv.org/abs/2409.07429>.
- Wei, J., Sun, Z., Papay, S., McKinney, S., Han, J., Fulford, I., Chung, H. W., Passos, A. T., Fedus, W., and Glaese, A. Browsecomp: A simple yet challenging benchmark for browsing agents. *arXiv preprint arXiv:2504.12516*, 2025a.
- Wei, J., Yang, Y., Zhang, X., Chen, Y., Zhuang, X., Gao, Z., Zhou, D., Wang, G., Gao, Z., Cao, J., Qiu, Z., He, X., Zhang, Q., You, C., Zheng, S., Ding, N., Ouyang, W., Dong, N., Cheng, Y., Sun, S., Bai, L., and Zhou, B. From ai for science to agentic science: A survey on autonomous scientific discovery, 2025b. URL <https://arxiv.org/abs/2508.14111>.
- Wen, L., Fu, D., Li, X., Cai, X., Ma, T., Cai, P., Dou, M., Shi, B., He, L., and Qiao, Y. Dilu: A knowledge-driven approach to autonomous driving with large language models, 2024. URL <https://arxiv.org/abs/2309.16292>.
- Wu, J., Yin, W., Jiang, Y., Wang, Z., Xi, Z., Fang, R., Zhang, L., He, Y., Zhou, D., Xie, P., and Huang, F. Web-walker: Benchmarking llms in web traversal, 2025a. URL <https://arxiv.org/abs/2501.07572>.
- Wu, Q., Bansal, G., Zhang, J., Wu, Y., Zhang, S., Zhu, E., Li, B., Jiang, L., Zhang, X., and Wang, C. Autogen: Enabling next-gen llm applications via multi-agent conversation framework, August 01, 2023 2023.
- Wu, R., Wang, X., Mei, J., Cai, P., Fu, D., Yang, C., Wen, L., Yang, X., Shen, Y., Wang, Y., and Shi, B. Evolver: Self-evolving llm agents through an experience-driven lifecycle, 2025b. URL <https://arxiv.org/abs/2510.16079>.

- Yang, C., Yang, X., Wen, L., Fu, D., Mei, J., Wu, R., Cai, P., Shen, Y., Deng, N., Shi, B., Qiao, Y., and Li, H. Learning on the job: An experience-driven self-evolving agent for long-horizon tasks, 2025. URL <https://arxiv.org/abs/2510.08002>.
- Ye, S., Yu, C., Ke, K., Xu, C., and Wei, Y. H2r: Hierarchical hindsight reflection for multi-task llm agents. *arXiv preprint arXiv:2509.12810*, 2025.
- Yin, Z., Sun, Q., Chang, C., Guo, Q., Dai, J., Huang, X., and Qiu, X. Exchange-of-thought: Enhancing large language model capabilities through cross-model communication, 2023. URL <https://arxiv.org/abs/2312.01823>.
- Zhang, G., Chen, K., Wan, G., Chang, H., Cheng, H., Wang, K., Hu, S., and Bai, L. Evoflow: Evolving diverse agentic workflows on the fly. *arXiv preprint arXiv:2502.07373*, 2025a.
- Zhang, G., Fu, M., Wan, G., Yu, M., Wang, K., and Yan, S. G-memory: Tracing hierarchical memory for multi-agent systems, 2025b. URL <https://arxiv.org/abs/2506.07398>.
- Zhang, G., Niu, L., Fang, J., Wang, K., Bai, L., and Wang, X. Multi-agent architecture search via agentic supernet. *arXiv preprint arXiv:2502.04180*, 2025c.
- Zhang, G., Wang, J., Chen, J., Zhou, W., Wang, K., and Yan, S. Agentracer: Who is inducing failure in the llm agentic systems?, 2025d. URL <https://arxiv.org/abs/2509.03312>.
- Zhang, J., Xiang, J., Yu, Z., Teng, F., Chen, X., Chen, J., Zhuge, M., Cheng, X., Hong, S., Wang, J., Zheng, B., Liu, B., Luo, Y., and Wu, C. AFlow: Automating Agentic Workflow Generation, October 2024a. URL <http://arxiv.org/abs/2410.10762>. arXiv:2410.10762.
- Zhang, J., Hu, S., Lu, C., Lange, R., and Clune, J. Darwin godel machine: Open-ended evolution of self-improving agents, 2025e. URL <https://arxiv.org/abs/2505.22954>.
- Zhang, W., Cui, C., Zhao, Y., Hu, R., Liu, Y., Zhou, Y., and An, B. Agentorchestra: A hierarchical multi-agent framework for general-purpose task solving, 2025f. URL <https://arxiv.org/abs/2506.12508>.
- Zhang, W., Li, X., Zhang, Y., Jia, P., Wang, Y., Guo, H., Liu, Y., and Zhao, X. Deep research: A survey of autonomous research agents, 2025g. URL <https://arxiv.org/abs/2508.12752>.
- Zhang, W., Zeng, L., Xiao, Y., Li, Y., Cui, C., Zhao, Y., Hu, R., Liu, Y., Zhou, Y., and An, B. Agentorchestra: Orchestrating hierarchical multi-agent intelligence with the tool-environment-agent(tea) protocol, 2025h. URL <https://arxiv.org/abs/2506.12508>.
- Zhang, Z., Bo, X., Ma, C., Li, R., Chen, X., Dai, Q., Zhu, J., Dong, Z., and Wen, J.-R. A survey on the memory mechanism of large language model based agents. *arXiv preprint arXiv:2404.13501*, 2024b.
- Zhang, Z., Dai, Q., Chen, X., Li, R., Li, Z., and Dong, Z. Memengine: A unified and modular library for developing advanced memory of llm-based agents, 2025i. URL <https://arxiv.org/abs/2505.02099>.
- Zhao, A., Huang, D., Xu, Q., Lin, M., Liu, Y.-J., and Huang, G. Expel: Llm agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pp. 19632–19642, 2024.
- Zhao, S., Zhang, H., Lin, S., Li, M., Wu, Q., Zhang, K., and Wei, C. Pyvision: Agentic vision with dynamic tooling, 2025. URL <https://arxiv.org/abs/2507.07998>.
- Zheng, B., Fatemi, M. Y., Jin, X., Wang, Z. Z., Gandhi, A., Song, Y., Gu, Y., Srinivasa, J., Liu, G., Neubig, G., and Su, Y. Skillweaver: Web agents can self-improve by discovering and honing skills, 2025. URL <https://arxiv.org/abs/2504.07079>.
- Zheng, L., Wang, R., Wang, X., and An, B. Synapse: Trajectory-as-exemplar prompting with memory for computer control. *arXiv preprint arXiv:2306.07863*, 2023.
- Zhong, W., Guo, L., Gao, Q., Ye, H., and Wang, Y. Memorybank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pp. 19724–19731, 2024.

A. EvolveLab Implementation

EvolveLab is designed as a modular and extensible codebase to support the systematic study of self-evolving agent memory systems. It provides a unified interface that abstracts the complexities of diverse memory architectures, enabling standardized implementation, evaluation, and meta-evolution.

A.1. Unified Interface and Abstract Base Class

The cornerstone of **EvolveLab** is the `BaseMemoryProvider` abstract base class (ABC), which defines the fundamental protocol for all memory systems. As shown in the code snippet below, the interface enforces two primary operations that map to the modular design space (*Encode*, *Store*, *Retrieve*, *Manage*):

- **Retrieve (`provide_memory`):** Handles context-aware memory recall. It accepts a `MemoryRequest` containing the current task query, execution context, and system status, and returns a `MemoryResponse` containing a list of relevant `MemoryItems`.
- **Encode & Store (`take_in_memory`):** Orchestrates the ingestion of new experiences. This method processes a `TrajectoryData` object, which encapsulates the complete history of a task execution, extracts structural insights or tools (*Encode*), and persists them into the underlying storage medium (*Store*).

While `take_in_memory` primarily integrates the *Encode* and *Store* stages, the *Manage* functionality that is responsible for offline consolidation or selective forgetting is typically implemented as auxiliary methods within the provider classes or invoked during specific lifecycle events.

```

1 class BaseMemoryProvider(ABC):
2     """Abstract base class for memory providers"""
3
4     def __init__(self, memory_type: MemoryType, config: Optional[dict] = None):
5         self.memory_type = memory_type
6         self.config = config or {}
7
8     @abstractmethod
9     def provide_memory(self, request: MemoryRequest) -> MemoryResponse:
10        """
11        Retrieve relevant memories based on query, context and status
12        Args:
13            request: MemoryRequest containing query, context, status and optional params
14        Returns:
15            MemoryResponse containing relevant memories
16        """
17        pass
18
19    @abstractmethod
20    def take_in_memory(self, trajectory_data: TrajectoryData) -> tuple[bool, str]:
21        """
22        Store/ingest new memory from trajectory data
23        Args:
24            trajectory_data: TrajectoryData containing query, trajectory and metadata
25        Returns:
26            tuple[bool, str]: (Success status of memory ingestion, Description of absorbed
27            memory)
28        """
29        pass
30
31    @abstractmethod
32    def initialize(self) -> bool:
33        """
34        Initialize the memory provider (load existing data, setup indices, etc.)
35        Returns:
36            bool: Success status of initialization
37        """
38        pass

```

```

38
39 def get_memory_type(self) -> MemoryType:
40     """Get the type of this memory provider"""
41     return self.memory_type
42
43 def get_config(self) -> dict:
44     """Get the configuration of this memory provider"""
45     return self.config.copy()

```

Listing 1. The Abstract Base Class of Memory Providers

A.2. Standardized Data Carriers

To ensure seamless interoperability across heterogeneous memory designs and agent frameworks, **EvoLab** utilizes standardized memory data carriers. These structures act as the "universal language" of the framework:

- **MemoryItem**: The fundamental unit of information, capable of representing raw text, distilled insights, or executable code (APIs). Each item includes metadata such as creation timestamps, confidence scores, and source identifiers.
- **TrajectoryData**: A comprehensive container for task execution history, including the initial query, full interaction traces (state-action pairs), and terminal rewards. It serves as the raw substrate for memory evolution.
- **MemoryRequest/Response**: Standardized envelopes for retrieval queries and results, ensuring that any agent system can interact with any memory provider without architecture-specific modifications.

A.3. Implementation Examples: ExpeL and SkillWeaver

The versatility of the **EvoLab** interface is demonstrated by our implementation of twelve distinct memory systems. Two representative examples are:

- **ExpeLProvider**: Implements a contrastive learning-based memory. Its `take_in_memory` function identifies successful and failed trajectories to distill high-level "insights" into a textual format. These insights are stored in a vector database and retrieved via semantic similarity during `provide_memory` to guide the agent away from previous mistakes.
- **SkillWeaverProvider**: Operates in a tool-centric design space. Its `take_in_memory` logic uses an LLM to synthesize reusable Python functions (skills) from successful trajectories. These skills are stored as executable code-level repositories and are dynamically retrieved and injected into the agent's action space through the unified `MemoryItem` interface.

A.4. Evaluation

Beyond unified implementation, **EvoLab** provides a standardized testbed for rigorously assessing memory architectures across diverse agentic tasks. The framework offers out-of-the-box support for multiple challenging benchmarks, including GAIA (Mialon et al., 2023), xBench (Chen et al., 2025), and WebWalkerQA (Wu et al., 2025a). **EvoLab** accommodates two evaluation paradigms: an **online** mode, where the experiential memory base is updated on-the-fly as the agent system processes a continuous stream of tasks, and an **offline** mode, where the memory system first accumulates experience from a static set of trajectories before being assessed on separate, unseen tasks. To ensure robust and versatile assessment, we support multiple evaluation protocols, including exact string matching and flexible LLM-as-a-Judge.

B. Experiment Details

B.1. Dataset Details

The four datasets used in this study are described and summarized as follows:

- **GAIA** (Mialon et al., 2023) consists of 165 tasks, including 53 Level-1, 86 Level-2, and 26 Level-3 problems. For evaluating **MemEvolve** on GAIA+🧠 and GAIA+⚡, the memory systems are evolved using GAIA Level-1 tasks together with 67 TaskCraft queries. Meta-evolution is conducted for three rounds, with 40 trajectories per round.

Table 5. Three-run robustness evaluation for MemEvo and competitive self-improving memory baselines in Table 3. We report mean accuracy and standard deviation.

Method	GAIA	WebWalkerQA	xBench
Voyager	69.90 $\pm$ 0.76	73.14 $\pm$ 1.00	68.00 $\pm$ 1.63
AWM	67.68 $\pm$ 1.51	72.16 $\pm$ 0.28	70.33 $\pm$ 0.47
Mobile-E	68.69 $\pm$ 1.03	71.76 $\pm$ 1.44	68.33 $\pm$ 1.25
MemEvo	73.94 $\pm$ 0.86	76.27 $\pm$ 1.47	73.67 $\pm$ 0.47

Table 6. Extended evolution beyond the default three rounds. The OOD WebWalkerQA column evaluates the best architecture from each round without re-evolution.

Round	TaskCraft	Cost	Delay	OOD WebWalkerQA
R0	61.67	0.092	485.25	65.10 $\pm$ 2.65
R1 best	70.00	0.105	512.40	67.06 $\pm$ 0.96
R2 best	75.00	0.073	355.40	70.20 $\pm$ 1.47
R3 best	78.33	0.056	364.37	76.27 $\pm$ 1.47
R4 best	80.00	0.048	318.52	76.86 $\pm$ 1.94
R5 best	80.00	0.043	295.60	77.65 $\pm$ 1.44

Table 7. Cross-backbone transfer with Claude-3.7-Sonnet. The MemEvo memory architecture is evolved on TaskCraft with GPT-5-Mini and transferred to Claude-3.7-Sonnet without re-evolution.

Framework	Model	WebWalkerQA	xBench-DS	TaskCraft	GAIA Avg.
OAgents	Claude-3.7-Sonnet	58.23	47.00	—	66.67
CK-Pro (pass@1)	Claude-3.7-Sonnet	60.64	56.00	66.00	60.00
CK-Pro (pass@3)	Claude-3.7-Sonnet	65.88	67.00	70.66	75.15
MemEvo +Flash (pass@1)	Claude-3.7-Sonnet	71.76	69.00	69.33	69.69
MemEvo +Flash (pass@3)	Claude-3.7-Sonnet	77.05	74.00	75.33	78.78

- *WebWalkerQA* (Wu et al., 2025a) evaluates an agent’s ability to handle complex, multi-turn web interactions, comprising 680 real-world queries across four domains and over 1,373 webpages. We sample a subset of 170 queries for evaluation, with the sampling script released in our codebase. All memory systems used for WebWalkerQA are meta-evolved on TaskCraft.
- *xBench-DeepSearch* (xBench-DS) (Chen et al., 2025) contains 100 tasks that assess agentic planning, tool use, and reasoning. Similar to WebWalkerQA, the memory systems used for xBench-DS evaluation are entirely meta-evolved on TaskCraft.
- *TaskCraft* (Shi et al., 2025a) is a synthetic benchmark generated via an autonomous data pipeline. We collect 300 queries as a working subset and use 120 of them for three rounds of meta-evolution, with 40 queries per round. Meta-evolution for 🧠 and ⚡ is performed independently.

B.2. Robustness Analyses

We report robustness analyses on repeated trials, longer evolution horizons, cross-backbone transfer, and search baselines.

Statistical Significance Testing We repeat representative evaluations three times for MemEvo and competitive memory baselines. As shown in Table 5, MemEvo maintains the highest mean performance across all three benchmarks with low cross-run variance.

Extended Evolution We extend the evolution horizon from three to five rounds to test sensitivity to $K_{\max}$. Table 6 shows that later rounds preserve the accuracy gains while further reducing cost and delay.

Cross-Backbone Transfer We evaluate transfer to Claude-3.7-Sonnet under the same fixed evolved memory architecture. As reported in Table 7, the transferred memory remains effective under this unseen backbone.

Search Baselines We compare diagnose-and-design evolution with random architecture composition and greedy component substitution. Table 8 indicates that structured diagnosis yields larger and more stable gains than both search baselines.

Table 8. Search-baseline comparison. All methods start from the same AgentKB initialization. Greedy Search enumerates eight single-module substitutions per round and accepts the locally best variant; MemEvolve uses trajectory-grounded diagnosis to generate three holistic descendants. WebWalkerQA is evaluated OOD without re-evolution.

Round	MemEvolve		Random Search		Greedy Search	
	TaskCraft	WebWalkerQA	TaskCraft	WebWalkerQA	TaskCraft	WebWalkerQA
R0	61.67	65.10	61.67	65.10	61.67	65.10
R1 best	70.00	67.06	63.33	64.51	65.00	65.88
R2 best	75.00	70.20	66.67	67.84	68.33	68.24
R3 best	78.33	76.27	65.00	65.88	70.00	70.59
Δ	+16.66	+11.17	+3.33	+0.78	+8.33	+5.49

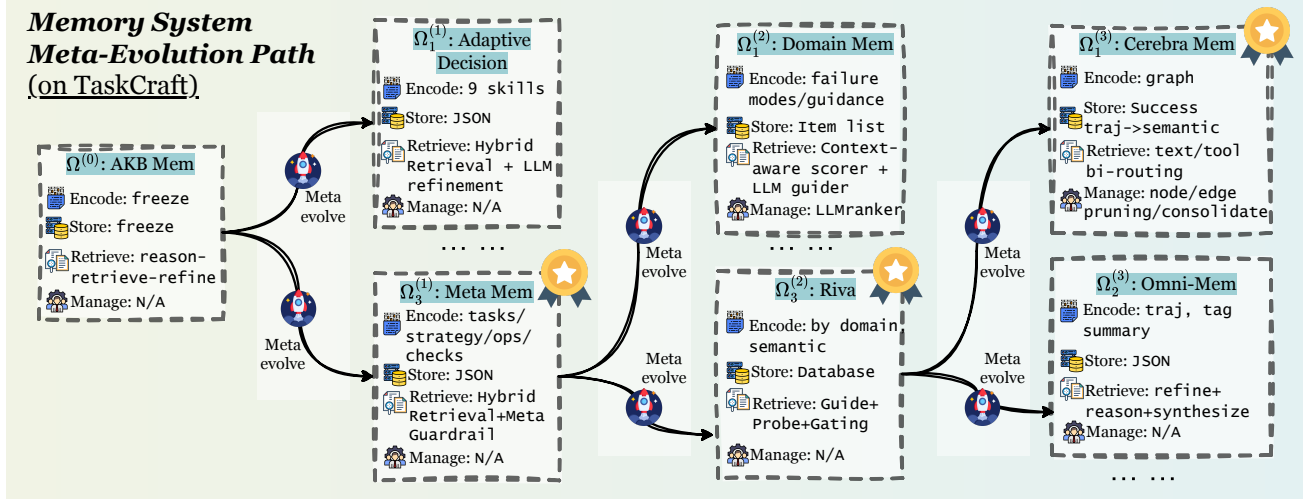


Figure 6. Illustration of the progressive evolution from the fixed AgentKB architecture to increasingly agentic and efficient memory architectures. Each stage reflects structural and functional modifications in memory encoding, storing, retrieval, and maintenance, culminating in high-performing systems such as Riva and Cerebra.

B.3. Evolution Dynamics

We illustrate the progressive evolution from the fixed AgentKB architecture to increasingly agentic and efficient memory architectures of MemEvolve in Figure 6.

C. Memory System Demonstration

To provide a concrete and intuitive understanding of the memory architectures evolved by MemEvolve, we visualize three representative systems discovered along different evolutionary trajectories, as shown in Figures 7 to 9. These examples highlight how MemEvolve progressively transforms simple, static memory mechanisms into more expressive and adaptive architectures by modifying memory encoding, retrieval, and management strategies. Together, they illustrate the diversity of memory designs that can emerge under the same meta-evolutionary framework.

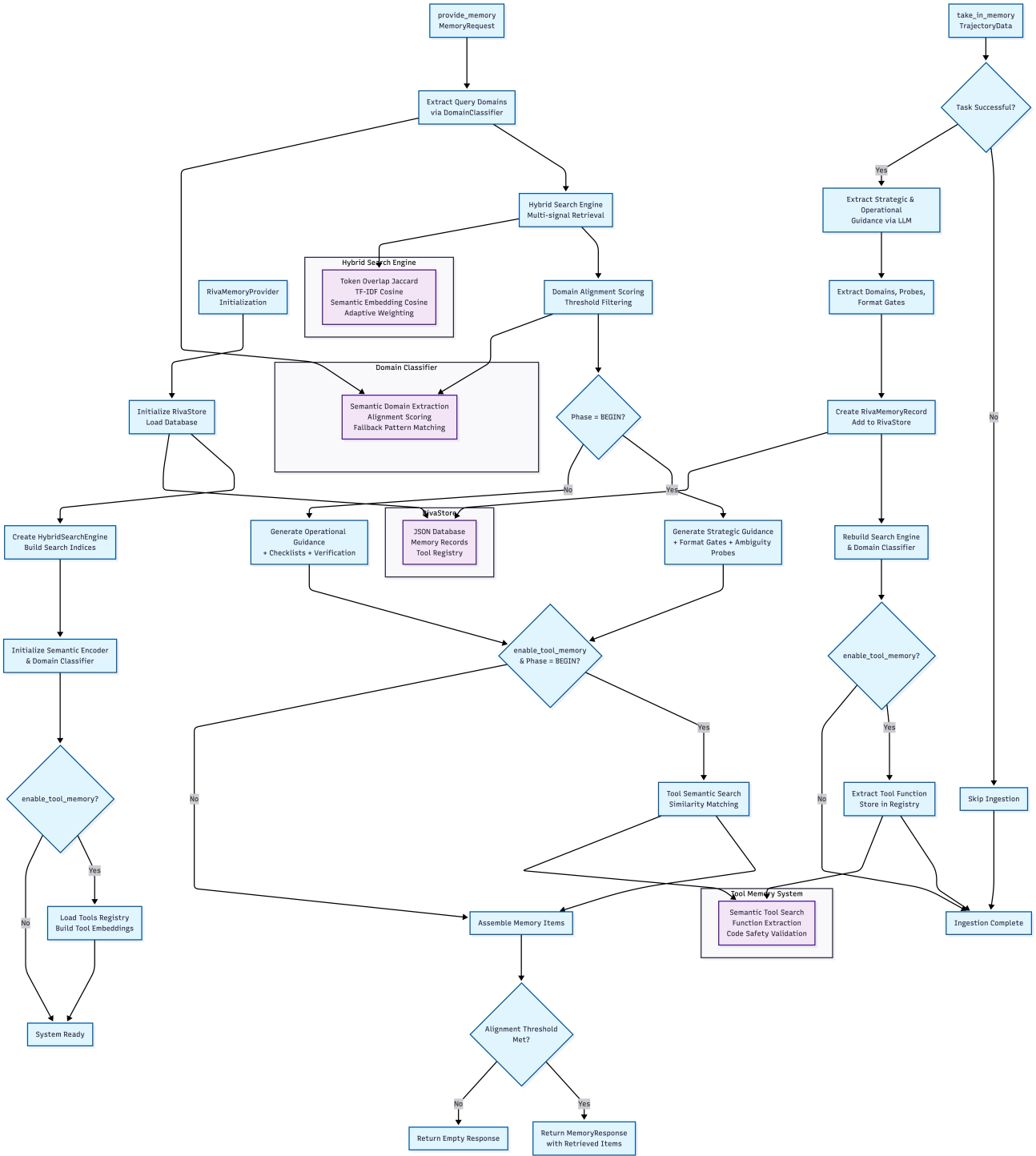


Figure 8. Illustration of the RIVA memory system evolved by MemEvolve. Its evolutionary initialization follows an AGENTKB-style architecture, but without inheriting the large and costly offline knowledge base. Through meta-evolution, RIVA develops more agent-centric encoding and retrieval strategies while remaining lightweight and fully online.

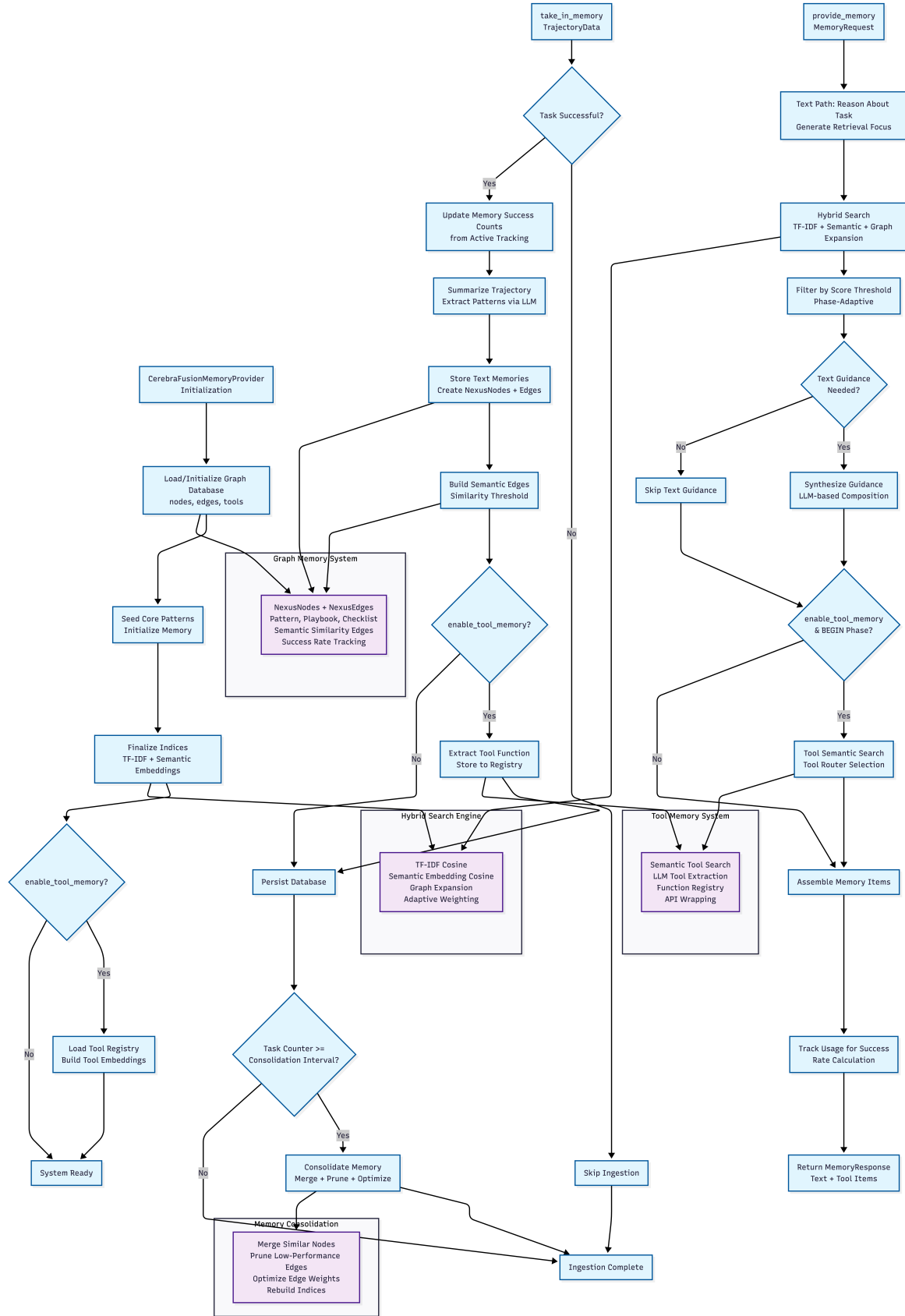


Figure 9. Illustration of the CEREBRA memory system evolved by MemEvo1ve. Starting from the same AGENTKB-style initialization (without the offline knowledge base), CEREBRA further evolves to distill both reusable tools and abstract knowledge from experience, and incorporates working memory maintenance mechanisms to support long-horizon agent evolution.